

Observability

Why observability

- Observability metrics are important because they give you numerical, real-time insight into your system's health and performance.
- To detect issues
- Without this can't find issues
- Detect high CPU or memory usage
- Identify pod failures
- Monitor node health
- Debug production issues quickly

What observability

- Observability metrics provides insights into the performance, behaviour, and health of application, system and infrastructure enabling observability practices, which is how a system's internal state is understood by examining its data.
- To collecting and analysing telemetry data – logs and traces and metrics.
- Organizations can obtain a comprehensive view of the performance of their technology stack, improving issue diagnostics and resolution times.

The 3 pillars of observability:

1. Metrics: (What is happening?)
 - Numerical data points collected from hardware, software, and websites.
 - used for monitoring resource usage, performance, and user behaviour.
 - It tells monitoring and observability teams **what** is happening in their systems.
 - Inside metrics again they defined into some core types such as

Application metrics:

- It is used in applications and within technology stack. These metrics allow engineers to monitor application performance and availability.

Examples: Response times, throughput, request rates, and error counts.

System metrics:

- System metrics, also referred to as infrastructure metrics, reflect the health of hardware and operating systems including key components like Kubernetes. These metrics provide insights into the performance of cloud resources, virtual machines, containers, and other underlying components.

Examples: CPU utilization, disk I/O, network throughput, memory usage

Business metrics:

- Business metrics tie technical and operational performance to business outcomes.
Example: conversion rates, average transaction value

2. Logs: (What exactly happened?)

- Logs are timestamped entries of specific events generated by systems, applications, networks, and infrastructure and text records of events. It allows engineers to why issues occur.
- **System logs** Show system activities like errors, connection attempts, and configuration changes.
- **Application logs** Show what the application is doing (user login, CRUD operations, errors, updates).
- **Network logs** Show network activities like traffic flow, security events, and user access.

3. Traces: (Where did it go wrong?)

- Traces allow engineers to monitor and debug applications. Traces tell Devops teams **where** issues are occurring in their environments.
- Traces that help identify slow processes include API queries, front-end API traffic, server-to-server workloads, and internal API calls.
- By analyzing traces, engineers can discover which metrics or logs are related to a particular issue, mitigating future issues.

SRE Goldens Rules:

Latency – Time taken to service a request.

Example: Swiggy app takes time to open menu loads

Traffic – The volume of requests or transactions.

Example: Huge number of orders per minute during lunch hour triggers auto-scaling

Errors – The number or rate of failed requests.

Example: Payment failed issues

Saturation – The extent to which system resources to used.

Example: Database issues